Status	Finished
Started	Tuesday, 22 September 2026, 3:26 PM
Completed	Tuesday, 22 September 2026, 3:35 PM
Duration	9 mins 46 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Find if a String2 is substring of String1. If it is, return the index of the first occurrence. else return -1.

Sample Input 1

thistest123string

123

Sample Output 1

8

Answer: (penalty regime: 0 %)

```
def find_substring_index(string1, string2):
    return string1.find(string2)

# Sample usage:
string1 = input().strip()
string2 = input().strip()

print(string1.find(string2))
```

	Input	Expected	Got	
✓	thistest123string 123	8	8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Given a **non-empty** string s and an abbreviation abbr, return whether the string matches with the given abbreviation.

A string such as "word" contains only the following valid abbreviations:

["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1", "1o2", "2r1", "3d", "w3", "4"]

Notice that only the above abbreviations are valid abbreviations of the string "word". Any other string is not a valid abbreviation of "word".

Note:

Assume s contains only lowercase letters and abbr contains only lowercase letters and digits.

Example 1:

Input

internationalization
i12iz4n

Output

true

Explanation

Given **s** = "internationalization", **abbr** = "i12iz4n":

Return true.

Example 2:

Input

apple
a2e

Output

false

Explanation

Given **s** = "apple", **abbr** = "a2e":

Return false.

Answer: (penalty regime: 0 %)

```
def validWordAbbreviation(s: str, abbr: str) -> bool:
    i = 0  # Pointer for string s
    j = 0  # Pointer for abbr

    while i < len(s) and j < len(abbr):
        if abbr[j].isdigit():
            # Leading zero in abbreviation numbers is inva
            if abbr[j] == '0':
                return False

            # Parse the complete number from abbr
            num = 0
            while j < len(abbr) and abbr[j].isdigit():
                num = num * 10 + int(abbr[j])
                j += 1

            # Advance pointer i by the parsed number
            i += num
        else:
            # Character match check
            if s[i] != abbr[j]:
                return False
            i += 1
            j += 1

    # Both pointers must reach the end of their respective
    return i == len(s) and j == len(abbr)

# Input processing
if __name__ == "__main__":
    import sys
    input_data = sys.stdin.read().split()
    if len(input_data) >= 2:
        s = input_data[0]
        abbr = input_data[1]
        result = validWordAbbreviation(s, abbr)
        print("true" if result else "false")
```

	Input	Expected	Got	
✓	internationalization i12iz4n	true	true	✓
✓	apple a2e	false	false	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

The program must accept **N** series of keystrokes as string values as the input. The character **^** represents undo action to clear the last entered keystroke. The program must print the string typed after applying the undo operations as the output. If there are no characters in the string then print **-1** as the output.

Boundary Condition(s):

1 <= N <= 100
1 <= Length of each string <= 100

Input Format:

The first line contains the integer N.
The next N lines contain a string on each line.

Output Format:

The first N lines contain the string after applying the undo operations.

Example Input/Output 1:

Input:

3
Hey ^ goooo^^glee^
lucke^y ^charr^ms
ora^^nge^^^^

Output:

Hey google
luckycharms
-1

Answer: (penalty regime: 0 %)

```
import sys

def process_keystrokes():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    n = int(input_data[0].strip())

    for k in range(1, n + 1):
        if k < len(input_data):
            line = input_data[k]
        else:
            line = ""

```

```
        stack = []
        for char in line:
            if char == '^':
                if stack:
                    stack.pop()
            else:
                stack.append(char)

        result = "".join(stack)
        if result:
            print(result)
        else:
            print(-1)

if __name__ == "__main__":
    process_keystrokes()
```

	Input	Expected	Got	
✓	3 Hey ^ goooo^^glee^ lucke^y ^charr^ms ora^^nge^^^^	Hey google luckycharms -1	Hey google luckycharms -1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

A pangram is a sentence where every letter of the English alphabet appears at least once.

Given a string sentence containing only lowercase English letters, return true if sentence is a pangram, or false otherwise.

Example 1:

Input:

thequickbrownfoxjumpsoverthelazydog

Output:

true

Explanation: sentence contains at least one of every letter of the English alphabet.

Example 2:

Input:

arvijayakumar

Output: false

Constraints:

1 <= sentence.length <= 1000

sentence consists of lowercase English letters.

For example:

Test	Result
`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`
`print(checkPangram('arvijayakumar'))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def checkPangram(sentence: str) -> str:
    if len(set(sentence)) == 26:
        return "true"
    else:
        return "false"

```

	Test	Expected	Got	
✓	`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`	`true`	✓
✓	`print(checkPangram('arvijayakumar'))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Given a string S which is of the format USERNAME@DOMAIN.EXTENSION, the program must print the EXTENSION, DOMAIN, USERNAME in the reverse order.

Input Format:

The first line contains S.

Output Format:

The first line contains EXTENSION.
The second line contains DOMAIN.
The third line contains USERNAME.

Boundary Condition:

1 <= Length of S <= 100

Example Input/Output 1:

Input:

abcd@gmail.com

Output:

com
gmail
abcd

For example:

Input	Result
arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar

Answer: (penalty regime: 0 %)

```
import sys


def parse_email():
    # Read email string from standard input
    s = sys.stdin.read().strip()

    if "@" in s:
        # Split into USERNAME and DOMAIN.EXTENSION at th
        username, rest = s.split("@", 1)

        # Split at the first '.' to separate DOMAIN from
        if "." in rest:
            domain, extension = rest.split(".", 1)

            # Print in reverse order: EXTENSION, DOMAIN,
            print(extension)
            print(domain)
            print(username)


if __name__ == "__main__":
    parse_email()
```

	Input	Expected	Got	
✓	abcd@gmail.com	com gmail abcd	com gmail abcd	✓
✓	arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar	edu.in rajalakshmi arvijayakumar	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Given a string, determine if it is a palindrome, considering only alphanumeric characters and ignoring cases.

Note: For the purpose of this problem, we define empty string as valid palindrome.

Example 1:

```
Input:
A man, a plan, a canal: Panama

Output:
1
```

Example 2:

```
Input:
race a car

Output:
0
```

Constraints:

- `s` consists only of printable ASCII characters.

Answer: (penalty regime: 0 %)

```
import sys


def isPalindrome(s: str) -> int:
    # Filter to keep only alphanumeric characters and conv
    filtered_chars = [char.lower() for char in s if char.i

    # Check if the filtered characters equal their reverse
    if filtered_chars == filtered_chars[::-1]:
        return 1
    else:
        return 0


# Main driver block for automated testing systems
if __name__ == "__main__":
    input_data = sys.stdin.read().strip()
    print(isPalindrome(input_data))
```

	Input	Expected	Got	
✓	A man, a plan, a canal: Panama	1	1	✓
✓	race a car	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Write a Python program to get one string and reverses a string. The input string is given as an array of characters `char[]`.

You may assume all the characters consist of printable ascii characters.

Example 1:

```
Input:
hello
Output:
olleh
```

Example 2:

```
Input:
Hannah
Output:
hannaH
```

Answer: (penalty regime: 0 %)

```
import sys


def reverseString(s: list[str]) -> None:
    # Reverse the array of characters in-place using two
    left, right = 0, len(s) - 1
    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1


if __name__ == "__main__":
    # Read the input string
    input_data = sys.stdin.read().splitlines()
    if input_data:
        # Convert input string into a list of single cha
        char_array = list(input_data[0].rstrip("\r\n"))

        # Reverse in-place
        reverseString(char_array)

        # Print the reversed character array joined back
        print("".join(char_array))
```

	Input	Expected	Got	
✓	hello	olleh	olleh	✓
✓	Hannah	hannaH	hannaH	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

Open brackets must be closed by the same type of brackets.

Open brackets must be closed in the correct order.

Constraints:

1 <= s.length <= 10^4

s consists of parentheses only '()[]{}'.

For example:

Test	Result
print(ValidParenthesis("()"))	true
print(ValidParenthesis("()[]{}"))	true
print(ValidParenthesis("(]"))	false

Answer: (penalty regime: 0 %)

Reset answer

```
def ValidParenthesis(s: str) -> str:
    stack = []
    # Mapping of closing brackets to their corresponding c
    mapping = {")": "(", "}": "{", "]": "["}

    for char in s:
        if char in mapping:
            # Pop the top element if stack is not empty, e
            top_element = stack.pop() if stack else "#"

            # If the popped element doesn't match the requ
            if mapping[char] != top_element:
                return "false"
        else:
            # It's an opening bracket, push onto stack
            stack.append(char)

    # Return "true" if all open brackets were properly mat
    return "true" if not stack else "false"
```

	Test	Expected	Got	
✓	print(ValidParenthesis("()"))	true	true	✓
✓	print(ValidParenthesis("()[]{}"))	true	true	✓
✓	print(ValidParenthesis("(]"))	false	false	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Assume that the given string has enough memory.

Don't use any extra space(IN-PLACE)

Sample Input 1

a2b4c6

Sample Output 1

aabbbbccccccc

Answer: (penalty regime: 0 %)

```
import sys


def expandString(s: str) -> str:
    result = []
    i = 0
    n = len(s)

    while i < n:
        char = s[i]
        i += 1

        # Extract the multi-digit number following the c
        count_str = ""
        while i < n and s[i].isdigit():
            count_str += s[i]
            i += 1

        # Multiply character by the count (default to 1
        count = int(count_str) if count_str else 1
        result.append(char * count)

    return "".join(result)


if __name__ == "__main__":
    input_data = sys.stdin.read().split()
    if input_data:
        print(expandString(input_data[0]))
```

	Input	Expected	Got	
✓	a2b4c6	aabbbbccccccc	aabbbbccccccc	✓
✓	a12b3d4	aaaaaaaaaaaabbbdddd	aaaaaaaaaaaabbbdddd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Consider the below words as key words and check the given input is key word or not.

keywords: {break, case, continue, default, defer, else, for, func, goto, if, map, range, return, struct, type, var}

Input format:

Take string as an input from stdin.

Output format:

Print the word is key word or not.

Example Input:

break

Output:

break is a keyword

Example Input:

IF

Output:

IF is not a keyword

For example:

Input	Result
break	break is a keyword
IF	IF is not a keyword

Answer: (penalty regime: 0 %)

```
import sys


def check_keyword():
    # Set of specified Go-style keywords
    keywords = {
        "break",
        "case",
        "continue",
        "default",
        "defer",
        "else",
        "for",
        "func",
        "goto",
        "if",
        "map",
        "range",
        "return",
        "struct",
        "type",
        "var",
    }

    # Read input string from standard input
    input_data = sys.stdin.read().split()

    if input_data:
        word = input_data[0]
        # Check case-sensitive presence in keywords set
```

```
        if word in keywords:
            print(f"{word} is a keyword")
        else:
            print(f"{word} is not a keyword")


if __name__ == "__main__":
    check_keyword()
```

	Input	Expected	Got	
✓	break	break is a keyword	break is a keyword	✓
✓	IF	IF is not a keyword	IF is not a keyword	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.